Project Report

Group 8

Abhishek Sharma
Ahmed Adnan

Ashutosh Singh
Hemakshi Bansal
Praneel Iralapalle
Prathamesh Kukade
Saurabh Raj
Saloni Kumari
Shivang Dikshit

Contents

1 Section 1: Digital Image Fundamentals 4
1.1 Digital Images as Signals ... 4
1.2 Spatial Domain vs Frequency Domain ... 4
1.3 Image Transforms ... 4
1.4 The Discrete Fourier Transform (DFT) ... 5
1.5 Fourier Spectrum: Magnitude and Phase ... 5
1.6 Fast Fourier Transform (FFT) ... 5
1.7 Frequency Domain Filters ... 5

2 Section 2: Histograms and Color Transforms 7
2.1 Histograms ... 7
2.2 HSV Color Space ... 7
2.3 Basic Image Adjustments ... 7
2.3.1 Brightness ... 7
2.3.2 Contrast ... 8
2.3.3 Saturation and Vibrance ... 8
2.3.4 Hue ... 8
2.4 Advanced Corrections ... 8
2.4.1 Gamma Correction ... 8
2.4.2 White Balance ... 8

3 Section 3: Convolution and Gradients 9
3.1 Convolution in Image Processing ... 9
3.2 Kernel Properties ... 9
3.3 Smoothing and Blurring ... 9
3.4 Image Gradients ... 10
3.5 The Sobel Operator ... 10
3.6 Canny Edge Detection ... 10
3.7 Image Sharpening ... 10

4 Section 4: Basic Machine Learning 11
4.1 Linear Regression ... 11
4.1.1 Optimization Methods ... 11
4.2 Logistic Regression ... 11
4.3 K-Means Clustering ... 12
4.4 Underfitting, Overfitting, and Regularization ... 12
4.4.1 Regularization ... 12

5 Section 5: CNN Architectures & Compression 13
5.1 Fully Connected Neural Networks (FCNN) ... 13
5.2 Convolutional Neural Networks (CNNs) ... 13
5.3 Pooling and Invariance ... 13
5.4 Deep Neural Network Compression ... 14
5.4.1 1. Pruning ... 14
5.4.2 2. Quantization ... 14
5.4.3 3. Clustering and Weight Sharing ... 14
5.5 PyTorch Fundamentals ... 14

6 Section 6: Dataset Acquisition & PyTorch Design 15
6.1 Dataset Acquisition from Kaggle 15
6.1.1 Why Kaggle is Used 15
6.1.2 Authentication Mechanism 15
6.1.3 Download and Extraction 15
6.2 Custom Dataset Design in PyTorch 15
6.2.1 The Dataset Abstraction 15
6.3 Internal Mechanics: What PyTorch Provides 16
6.4 The Three Core Methods 16
6.4.1 1. __init__: Construction 16
6.4.2 2. __len__: Cardinality 16
6.4.3 3. __getitem__: Execution 16

1 Section 1: Digital Image Fundamentals

Introduction

In the first section, we were introduced to the fundamental concepts required for understanding digital images and their mathematical representation. The session focused on building a conceptual understanding of how images are stored, how they can be analyzed, and why mathematical tools are required to extract meaningful information from them.

We learned that an image is not just a visual object but a signal that can be processed mathematically. The section emphasized the importance of analyzing images not only in the spatial domain but also in the frequency domain, where patterns, edges, and structures become easier to interpret.

1.1 Digital Images as Signals

A digital image is a discrete two-dimensional signal represented as a matrix of pixel values.

- **Grayscale Image:** Each pixel contains a single intensity value:

$$I(x, y) \in [0, 255]$$

- **Colour Image:** Colour images consist of multiple channels (R,G,B), each of which can be treated as an independent signal.

Images represented as matrices allow mathematical operations to be applied easily.

1.2 Spatial Domain vs Frequency Domain

- **Spatial Domain:** Represents images directly using pixel intensities.
- **Frequency Domain:** Represents how pixel intensities vary across space.

Key Differences

- **Low frequencies** correspond to smooth regions and illumination.
- **High frequencies** correspond to edges, noise, and fine details.
- Many image characteristics are easier to analyze in the frequency domain.

1.3 Image Transforms

Image transforms are mathematical tools that convert images from the spatial domain to the frequency domain.

Purpose of Image Transforms:

- Analyze image content in terms of frequency components.
- Separate important image features not obvious in the spatial domain.

Advantages:

- Isolate critical components of image patterns.
- Provide compact representation for storage and transmission.
- Make convolution and correlation computationally efficient.
- Are reversible using inverse transforms.

1.4 The Discrete Fourier Transform (DFT)

Since digital images are discrete, the Discrete Fourier Transform (DFT) is used for frequency analysis.

1D DFT Formula

For a discrete signal $f(x)$ of length N:

$$F(k) = \sum_{x=0}^{N-1} f(x)\, e^{-j2\pi kx/N}$$

where $k = 0, 1, 2, \ldots, N-1$, j is the imaginary unit, and $F(k)$ represents frequency components.

Inverse Discrete Fourier Transform (IDFT): The original signal can be reconstructed using the inverse DFT:

$$f(x) = \frac{1}{N} \sum_{k=0}^{N-1} F(k)\, e^{j2\pi kx/N}$$

This demonstrates that the DFT is reversible and information preserving.

Extension to Images: Images are two-dimensional signals. DFT is applied along both rows and columns.

1.5 Fourier Spectrum: Magnitude and Phase

The Fourier transform produces complex-valued output.

- **Magnitude:** $|F(k)| = \sqrt{\Re(F)^2 + \Im(F)^2}$
- **Phase:** $\theta(k) = \tan^{-1} \frac{\Im(F)}{\Re(F)}$

Note: Magnitude mainly affects texture and contrast, while phase preserves structural information such as edges and shapes.

To visualize the spectrum, logarithmic scaling is applied:

$$\text{Magnitude Spectrum} = 20 \cdot \log(1 + |F(u, v)|)$$

1.6 Fast Fourier Transform (FFT)

Direct computation of the DFT is computationally expensive ($O(N^2)$). The FFT reduces complexity to $O(N \log N)$, computing the same result significantly faster.

1.7 Frequency Domain Filters

Filtering modifies the Fourier transform of an image:

$$G(u, v) = F(u, v) \cdot H(u, v)$$

The distance from the center is given by $D(u, v) = \sqrt{(u - u_0)^2 + (v - v_0)^2}$.

Filter Types

1. Low Pass Filter (LPF): Preserves smooth regions, removes edges (blurring).

$$H_{LPF}(u,v) = \begin{cases} 1, & D(u,v) \leq D_0 \\ 0, & D(u,v) > D_0 \end{cases}$$

2. High Pass Filter (HPF): Preserves edges, suppresses background.

$$H_{HPF}(u,v) = 1 - H_{LPF}(u,v)$$

Conclusion

Session 1 introduced the mathematical foundation of image processing by treating images as discrete signals. These concepts are practical implementations using Fourier transforms and frequency-domain filtering.

2 Section 2: Histograms and Color Transforms

2.1 Histograms

A histogram is a graphical representation of the distribution of pixel intensities in an image. The x-axis represents the intensity values ranging from 0 (black) to 255 (white), while the y-axis represents the number of pixels corresponding to each intensity value.

Histograms provide a global view of the brightness and contrast characteristics of an image.

Interpreting Histogram Shapes

- **Left-leaning:** Indicates a dark (underexposed) image.
- **Centered:** Indicates balanced exposure.
- **Right-leaning:** Indicates a bright (overexposed) image.
- **Narrow width:** Indicates low contrast.
- **Wide width:** Indicates high contrast.

Types of Histograms

1. **Grayscale Histogram:** In a grayscale image, each pixel contains only intensity information. A single histogram is sufficient to analyze brightness distribution and contrast.
2. **RGB Histogram:** For color images, separate histograms are computed for Red, Green, and Blue channels.
    - Dominance of high **Red** values → Warmer image.
    - Dominance of high **Blue** values → Colder image.
    - Unequal peaks across channels indicate color imbalance.

2.2 HSV Color Space

HSV (Hue, Saturation, Value) is a color space designed to align more closely with human perception than RGB.

- **Hue:** Represents the pure color (angle on the color wheel).
- **Saturation:** Represents the purity or vividness of the color.
- **Value:** Represents the brightness or lightness of a pixel.

Note: In OpenCV, Hue values are typically scaled between 0 and 180 (to fit into 8-bit storage) instead of the standard 0 to 360 degrees.

2.3 Basic Image Adjustments

2.3.1 Brightness

Brightness refers to the overall luminance. Adjustment is performed by adding a constant bias β:

$$\text{new} = \text{old} + \beta$$

- Shifts the histogram left or right.
- **Risk:** Excessive brightness causes highlight clipping; low brightness causes shadow clipping.

2.3.2 Contrast

Contrast controls the difference in brightness between regions (how "distinct" the image looks).

$$\text{new} = \alpha \times \text{old}$$

Contrast Factors

- $\alpha > 1$: **Increases contrast** (stretches the histogram).
- $\alpha < 1$: **Decreases contrast** (compresses the histogram).

2.3.3 Saturation and Vibrance

- **Saturation:** Boosts intensity of *all* colors equally. Can lead to unnatural skin tones.
- **Vibrance:** Selectively enhances low-saturation pixels while protecting already saturated colors and skin tones. Results in a more natural look.

2.3.4 Hue

Hue manipulation rotates the color wheel. It changes the perceived color identity (e.g., Red → Green) while preserving Brightness, Saturation, and Texture.

2.4 Advanced Corrections

2.4.1 Gamma Correction

Gamma correction accounts for the non-linear response of human vision. Images are stored in a gamma-compressed form.

$$V_{\text{out}} = V_{\text{in}}^{1/\gamma}$$

For standard CRT/Monitors, the gamma value is usually $\gamma \approx 2.2$.

2.4.2 White Balance

Corrects color casts so that white objects appear white regardless of lighting.

- **Gray World Assumption:** Assumes the average color of the scene is gray.
- **White Patch Method:** Assumes the brightest pixel is white and scales channels accordingly.

3 Section 3: Convolution and Gradients

Introduction

This session covers the fundamental concepts of Computer Vision, focusing on how mathematical operations applied to digital images produce meaningful visual transformations. The core mechanism for this is **Convolution**.

3.1 Convolution in Image Processing

Convolution operates as a "sliding window" technique where a small matrix, known as a **Kernel**, moves over the image. At each position, the kernel's values are multiplied by the underlying image pixels and summed to produce a new pixel value.

Convolution Definition

Given an image $I(x, y)$ and a kernel $K(i, j)$, convolution is defined as:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j)$$

By altering the kernel values, we can perform operations such as blurring, sharpening, and edge detection.

3.2 Kernel Properties

The sum of all kernel elements determines the behavior of brightness in the output image:

- **Blur Kernels:** Typically sum to **1**. This ensures the average brightness of the image remains constant.
- **Edge Detectors:** Typically sum to **0**. This filters out constant regions (solid colors become black), effectively highlighting only the changes (edges).
- **Sharpening:** Typically sum to > 1 (or have negative weights) to amplify details.

3.3 Smoothing and Blurring

We explored two primary methods for noise reduction:

1. Average Blur (Box Blur): All pixels in the kernel have equal weight.

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Drawback: While effective at smoothing, it treats distant pixels the same as near ones, often resulting in "blocky" artifacts that degrade image detail.

2. Gaussian Blur: The kernel weights follow a Gaussian distribution (bell curve). The center pixel has the highest weight, with influence decreasing smoothly towards the edges.

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Advantage: This mimics the natural defocus of the human eye and preserves image structure better than average blur.

3.4 Image Gradients

Edges are regions with rapid changes in pixel intensity. We detect these using **Gradients**.

$$G_x = \frac{\partial I}{\partial x}, \quad G_y = \frac{\partial I}{\partial y}$$

- **Magnitude (S):** Represents the strength of the edge. $S = \sqrt{G_x^2 + G_y^2}$
- **Direction (θ):** Indicates the orientation of the edge. $\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$

3.5 The Sobel Operator

The Sobel operator computes gradients using discrete convolution kernels.

Sobel Kernels

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

3.6 Canny Edge Detection

The Canny detector is a robust, multi-stage algorithm designed to detect accurate, thin, and continuous edges:

1. **Gaussian Smoothing:** Reduce noise.
2. **Gradient Computation:** Use Sobel operators.
3. **Non-Maximum Suppression:** Thin the edges to 1-pixel width.
4. **Double Thresholding:** Identify strong vs. weak edges.
5. **Hysteresis:** Connect weak edges only if they touch strong edges.

3.7 Image Sharpening

Sharpening emphasizes texture and boundaries.

1. Laplacian Sharpening: Utilizes the second derivative ($\nabla^2 I$) to highlight regions of rapid change.

$$I_{\text{sharp}} = I - \alpha \nabla^2 I$$

2. Unsharp Masking: A controlled approach involving three steps:

- **Step 1:** Create a blurred version of the image (B).
- **Step 2:** Subtract the blur from the original to create a "mask" of high-frequency details ($M = I - B$).
- **Step 3:** Add this mask back to the original image ($I_{\text{sharp}} = I + kM$), where k controls the amount of sharpening.

Conclusion

Convolution in the spatial domain corresponds to multiplication in the frequency domain. Low frequencies represent smooth regions, while high frequencies represent edges. Thus, convolution kernels act as frequency-domain masks.

4 Section 4: Basic Machine Learning

Introduction

We learned the basics of Machine Learning and the pure mathematics behind how to manually apply Linear Regression, Logistic Regression, and K-Means Clustering.

4.1 Linear Regression

Target: Used to model a linear relationship between input features $\mathbf{x}$ and a continuous target variable y:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector and b is the bias term.

Loss Function (MSE): The Mean Squared Error (MSE) loss is minimized:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

4.1.1 Optimization Methods

There are two main ways to optimize weights: iterative (Gradient Descent) and analytical (Normal Equation).

Optimization Equations

1. Gradient Descent (Iterative):

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

This updates weights $\mathbf{w}$ using the learning rate η.

2. Normal Equation (Analytical):

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Evaluation: The coefficient of determination R^2 is used:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

4.2 Logistic Regression

Used for binary classification. Probabilities are modeled using the sigmoid function.

Binary Classification Model

Sigmoid Function:

$$p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

Binary Cross-Entropy Loss:

$$L = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Multiclass Extension: For more than two classes, we use the Softmax function:

$$p(y = k|\mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_j e^{\mathbf{w}_j^\top \mathbf{x}}}$$

4.3 K-Means Clustering

K-Means is an unsupervised learning algorithm used to group similar, unlabeled data points into k distinct clusters.

Objective Function:

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Algorithm Steps:

1. Assign points to the nearest centroid.
2. Update centroids as the mean of the assigned points.
3. Repeat until convergence.

4.4 Underfitting, Overfitting, and Regularization

1. Underfitting: Occurs when a model is too simple (high bias). It performs poorly on both training and test data. Causes include insufficient complexity or excessive regularization.

2. Overfitting: Occurs when a model learns noise in the training data (high variance). It has low training error but poor performance on unseen data. Causes include high-degree polynomials or deep networks without constraints.

4.4.1 Regularization

Regularization controls model complexity to improve generalization by penalizing large parameter values.

Regularization Techniques

L2 Regularization (Ridge Regression): Adds a squared penalty on weights. Shrinks weights smoothly toward zero.

$$L_{\text{L2}} = \text{MSE} + \lambda \|\mathbf{w}\|_2^2$$

L1 Regularization (Lasso Regression): Adds an absolute-value penalty. Promotes sparsity (sets some weights to zero).

$$L_{\text{L1}} = \text{MSE} + \lambda \|\mathbf{w}\|_1$$

5 Section 5: CNN Architectures & Compression

Introduction

Architectures such as Convolutional Neural Networks (CNNs) are capable of learning complex representations directly from data. However, as these models grow deeper, they introduce challenges related to computation cost, memory usage, and deployment. This section covers CNNs, compression techniques, and PyTorch fundamentals.

5.1 Fully Connected Neural Networks (FCNN)

An artificial neural network where every neuron in one layer is connected to every neuron in the subsequent layer.

FCNN Math

For a single layer:

$$\mathbf{A} = \sigma(\mathbf{WX} + \mathbf{b})$$

where $\mathbf{W}$ is the weight matrix and $\mathbf{b}$ is the bias.

Limitations:

- Huge parameter count for high-dimensional inputs.
- High memory requirements.
- Lack of spatial awareness (inefficient for images).

5.2 Convolutional Neural Networks (CNNs)

CNNs are designed for grid-structured data (images), leveraging local connectivity and weight sharing.

[Image of CNN architecture diagram]

Convolution Operation

The convolution applies a kernel K over image X:

$$Y(i,j) = \sum_{m,n} X(i+m, j+n) \cdot K(m,n)$$

This detects local spatial patterns while preserving position.

CNN Pipeline:

1. Convolution → 2. ReLU Activation → 3. Pooling → 4. Flatten → 5. Fully Connected → 6. Softmax

5.3 Pooling and Invariance

Pooling reduces spatial resolution to lower complexity and improve generalization.

- **Max Pooling:** Selects the maximum value in a window. Provides robustness to small translations.
- **Invariance:** CNNs offer translation invariance and robustness to local deformations.

5.4 Deep Neural Network Compression

Techniques to reduce model size for deployment on edge devices.

5.4.1 1. Pruning

Removes redundant or less important weights.

$$|w| < \tau \Rightarrow w = 0$$

Result: Sparse matrices (CSC/CSR formats), lower memory/compute costs.

5.4.2 2. Quantization

Reduces numerical precision (e.g., Float32 → Int8).

$$\text{Memory Ratio} \approx \frac{(n \times n) \times 8 + n \times 32}{(n \times n) \times 32}$$

Up to 4× reduction in memory usage.

5.4.3 3. Clustering and Weight Sharing

Weights are grouped (e.g., K-means); each group shares a representative value. Gradients are accumulated for the shared value during training.

5.5 PyTorch Fundamentals

Tensors: The core data structure.

- 0D (Scalar), 1D (Vector), 2D (Matrix), 3D+ (Images/Batches).
- Support GPU acceleration for massive parallelism.

Automatic Differentiation (Autograd): PyTorch builds a Directed Acyclic Graph (DAG) during the forward pass to record operations.

- `loss.backward()`: Computes gradients automatically.
- `torch.no_grad()`: Disables gradient tracking (inference mode).

Training Workflow

1. Load/Preprocess data (DataLoaders).
2. **Forward Pass:** Compute predictions.
3. **Loss Computation:** Compare with ground truth.
4. **Backpropagation:** `loss.backward()`.
5. **Optimizer Step:** Update weights (SGD/Adam).

Conclusion

CNNs exploit spatial structure for efficient vision tasks. Compression techniques (Pruning, Quantization) enable real-world deployment, while PyTorch simplifies development via automatic differentiation.

6 Section 6: Dataset Acquisition & PyTorch Design

6.1 Dataset Acquisition from Kaggle

6.1.1 Why Kaggle is Used

Kaggle provides curated datasets, stable directory structures, and reproducible access via API. Instead of manually uploading data, the Kaggle API allows automated dataset retrieval inside a notebook or remote environment.

6.1.2 Authentication Mechanism

Kaggle authentication is based on an API token stored in a file: kaggle.json. This file contains the Username and API key.

Environment Binding:

```
os.environ['KAGGLE_CONFIG_DIR'] = "/content"
```

This tells the Kaggle CLI where to look for the authentication file (default is ~/.kaggle, which may not persist in cloud environments).

Permission Hardening:

```
chmod 600 kaggle.json
```

- **Owner:** Read + Write.
- **Group/Others:** No access.
- *Reasoning:* Prevents credential leakage; Kaggle CLI refuses to run if permissions are too open.

6.1.3 Download and Extraction

Command:

```
kaggle datasets download -d moltean/fruits
```

Extraction:

```
unzip fruits.zip -d /content/fruits_dataset
```

Explicit extraction paths avoid polluting the working directory and provide a predictable root for traversal.

6.2 Custom Dataset Design in PyTorch

6.2.1 The Dataset Abstraction

In machine learning, raw data rarely exists in a form directly consumable by models. PyTorch introduces the Dataset abstraction to decouple data storage from model logic.

The Dataset Contract

torch.utils.data.Dataset is an **abstract interface.** It enforces a simple contract:

> "If you can tell me how many samples you have, and how to retrieve the i-th sample, I can handle everything else."

This is enforced via two required methods:

$$\_\_len\_\_(), \quad \_\_getitem\_\_(i)$$

Key Insight: PyTorch never inspects your data directly. It only calls these two functions.

6.3 Internal Mechanics: What PyTorch Provides

Once a class satisfies the Dataset contract, PyTorch automatically handles:

- **Index-Based Sampling:** PyTorch assumes sample $=$ dataset[i].
- **Automatic Batching:** The DataLoader recursively stacks samples into tensors: $X = \text{stack}(x_1, \ldots, x_B)$.
- **Shuffling:** Generates a random permutation π and accesses dataset[$\pi(i)$].
- **Parallel Loading:** If num_workers $>$ 0, PyTorch forks worker processes to run __getitem__ in parallel.

6.4 The Three Core Methods

6.4.1 1. __init__: Construction

```
def __init__(self, root_dir)
```

Purpose: Discover data locations, build label mappings, and perform one-time setup.

- Collects image file paths.
- Infers class names from directories.
- Creates cls_to_idx mapping.

6.4.2 2. __len__: Cardinality

```
def __len__(self)
```

Returns the total number of samples (N). Used by PyTorch for epoch size calculation and progress bars. **Must be deterministic.**

6.4.3 3. __getitem__: Execution

```
def __getitem__(self, idx)
```

This defines the **actual data pipeline**.

- **Input:** An index i.
- **Action:** Load image, apply preprocessing (Canny, LBP), convert to Tensor.
- **Output:** A dictionary containing the image, features, and label.

Why this design is better

Approach	Scalability	Parallelism
Manual loops	Low	None
Custom loader	Medium	Manual
Dataset + DataLoader	**High**	**Automatic**